

Praktikum Arsitektur Proyek Go: Membangun Backend Aplikasi E-Commerce Terstruktur & Multi-Library

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Praktikum ini melengkapi materi teori pada day6-arsitektur-go.md. Kita akan membangun backend sistem kasir e-commerce modular bernama `go-shop` menggunakan tata letak flat package yang bersih di root proyek tanpa folder pembungkus `cmd` dan `internal`.

Untuk membuat aplikasi lebih tangguh dan realistis, kita akan mengintegrasikan beberapa pustaka (*library*) pihak ketiga populer:

1. `github.com/joho/godotenv` : Untuk membaca konfigurasi sistem (seperti port aplikasi dan token rahasia) dari berkas `.env`.
2. `github.com/google/uuid` : Untuk menghasilkan ID transaksi belanja (*Order ID*) yang unik secara otomatis.
3. `github.com/fatih/color` : Untuk mencetak log audit keuangan berwarna di terminal konsol (warna hijau untuk transaksi sukses, merah untuk error, dan kuning untuk peringatan).

Sesi 1: Setup Struktur Direktori & Dependensi (30 Menit)

1. Membuat Struktur Folder Proyek

Buka terminal dan buatlah tata letak direktori proyek berikut secara langsung di root proyek:

```
/go-shop
├── go.mod
├── .env                # Berkas konfigurasi env
├── main.go            # Entry point utama aplikasi kasir (di root)
├── auth
│   └── auth.go        # Paket keamanan token merchant
├── logger
│   └── logger.go      # Paket audit log berwarna
├── model
│   └── product.go     # Paket skema model produk
└── service
    └── cart.go        # Paket logika bisnis transaksi belanja
```

2. Inisialisasi Modul & Instalasi Pustaka Pihak Ketiga

Buka terminal di dalam direktori root `/go-shop` dan jalankan perintah-perintah berikut:

```
# 1. Inisialisasi modul Go dengan nama go-shop
go mod init go-shop

# 2. Unduh pustaka-pustaka eksternal yang dibutuhkan
go get github.com/joho/godotenv
go get github.com/google/uuid
go get github.com/fatih/color

# 3. Sinkronisasikan go.mod dan go.sum
go mod tidy
```

Sesi 2: Menulis Konfigurasi & Paket Keamanan (45 Menit)

1. Membuat Berkas Konfigurasi Lingkungan (`.env`)

Buatlah berkas bernama `.env` di direktori root `/go-shop` (sejajar dengan `go.mod`), lalu tambahkan baris konfigurasi berikut:

```
APP_PORT=8080
APP_ENV=development
MERCHANT_SECRET_TOKEN=rahasia-kasir-goshop-2026
```

2. Implementasi Paket Otorisasi Merchant (`auth/auth.go`)

Paket ini bertugas memvalidasi token kasir merchant sebelum diizinkan mengakses transaksi. Kita menggunakan *unexported field* `secretKey` agar token tidak dapat dimodifikasi langsung oleh package luar.

```
package auth

import "strings"

// Authenticator mendefinisikan interface kontrak otorisasi (Exported)
type Authenticator interface {
    ValidateToken(token string) bool
}

// merchantAuth adalah struct konkret (Unexported/Private)
type merchantAuth struct {
    secretKey string // Unexported field
}

// NewAuthenticator berfungsi sebagai Constructor Function (Exported)
func NewAuthenticator(key string) Authenticator {
    return &merchantAuth{
        secretKey: strings.TrimSpace(key),
    }
}

// ValidateToken mengimplementasikan interface Authenticator
func (ma *merchantAuth) ValidateToken(token string) bool {
    return ma.secretKey == strings.TrimSpace(token)
}
```

Sesi 3: Implementasi Paket Audit Log & Model Produk (45 Menit)

1. Implementasi Paket Logger Berwarna (`logger/logger.go`)

Paket ini menggunakan pustaka `fatih/color` untuk mewarnai teks di terminal dan menggunakan fungsi spesial `init()` untuk mengumumkan kesiapannya saat sistem backend dinyalakan.

```

package logger

import (
    "fmt"
    "time"

    "github.com/fatih/color"
)

// Inisialisasi variabel logger dengan formatting warna
var (
    greenPrint = color.New(color.FgGreen, color.Bold).PrintlnFunc()
    redPrint   = color.New(color.FgRed, color.Bold).PrintlnFunc()
    yellowPrint = color.New(color.FgYellow).PrintlnFunc()
)

// Fungsi init otomatis dieksekusi sekali saat paket di-load pertama kali
func init() {
    yellowPrint("[AUDIT SYSTEM] Menginisialisasi modul pencatatan transaksi terpusat")
}

// LogSuccess mencetak audit sukses berwarna hijau (Exported)
func LogSuccess(orderID string, amount float64) {
    timestamp := time.Now().Format("2006-01-02 15:04:05")
    greenPrint(fmt.Sprintf("[AUDIT SUCCESS - %s] Order %s berhasil diproses. Nilai: %s", timestamp, orderID, amount))
}

// LogFailure mencetak audit gagal berwarna merah (Exported)
func LogFailure(reason string) {
    timestamp := time.Now().Format("2006-01-02 15:04:05")
    redPrint(fmt.Sprintf("[AUDIT FAILURE - %s] Transaksi ditolak. Alasan: %s", timestamp, reason))
}

```

2. Implementasi Model Produk (`model/product.go`)

Mendefinisikan skema produk e-commerce dan logika validasi datanya.

```
package model

import "errors"

type Product struct {
    SKU    string
    Name   string
    Price  float64
    Stock  int
}

// Validate memeriksa kelayakan data produk
func (p *Product) Validate() error {
    if p.SKU == "" || p.Name == "" {
        return errors.New("SKU dan nama produk tidak boleh kosong")
    }
    if p.Price <= 0 {
        return errors.New("harga produk harus lebih dari nol")
    }
    if p.Stock < 0 {
        return errors.New("stok produk tidak boleh bernilai negatif")
    }
    return nil
}
```

Sesi 4: Paket Transaksi & Entry Point Utama (`main.go`) (60 Menit)

1. Implementasi Paket Keranjang Belanja (`service/cart.go`)

Paket ini menggunakan pustaka `google/uuid` untuk menghasilkan *Order ID* transaksi kasir yang unik secara acak.

```

package service

import (
    "errors"
    "fmt"
    "go-shop/model" // Diimport langsung dari folder root/model

    "github.com/google/uuid"
)

type CartService struct {
    Items map[string]int // Map: SKU → Kuantitas beli
}

// NewCartService menginisialisasi keranjang belanja baru
func NewCartService() *CartService {
    return &CartService{
        Items: make(map[string]int),
    }
}

// AddItem memasukkan item belanja ke keranjang dengan validasi stok
func (c *CartService) AddItem(prod model.Product, qty int) error {
    if err := prod.Validate(); err != nil {
        return err
    }
    if qty ≤ 0 {
        return errors.New("kuantitas belanja harus minimal 1 unit")
    }
    if prod.Stock < qty {
        return fmt.Errorf("stok produk '%s' tidak cukup (Sisa: %d)", prod.Name,
        )

        c.Items[prod.SKU] += qty
        return nil
    }

    // Checkout memproses total belanja dan menerbitkan Order ID UUID baru
    func (c *CartService) Checkout(products map[string]model.Product) (string, float64, error) {
        if len(c.Items) = 0 {
            return "", 0, errors.New("keranjang belanja masih kosong")
        }

        var totalAmount float64
        for sku, qty := range c.Items {
            prod, exists := products[sku]
            if !exists {
                return "", 0, fmt.Errorf("produk dengan SKU %s tidak ditemukan d
            }
            totalAmount += prod.Price * float64(qty)
        }
    }
}

```

```
// Generate UUID unik untuk Order ID
orderId := uuid.New().String()

// Kosongkan keranjang belanja setelah checkout sukses
c.Items = make(map[string]int)

return orderId, totalAmount, nil
}
```

2. Membuat Entry Point Utama (`main.go` di Root)

Berkas ini berada langsung di direktori root `/go-shop` . Berkas ini memuat berkas konfigurasi `.env` menggunakan library `godotenv` , merakit paket-paket modular, memvalidasi otorisasi token kasir, dan menjalankan simulasi checkout belanja.

```

package main

import (
    "fmt"
    "os"

    "go-shop/auth"
    "go-shop/logger"
    "go-shop/model"
    "go-shop/service"

    "github.com/joho/godotenv"
)

func main() {
    // 1. Load berkas .env dari root direktori
    errEnv := godotenv.Load()
    if errEnv != nil {
        fmt.Println("[WARNING] Gagal memuat berkas .env, menggunakan variabel si
    }

    // Membaca variabel env
    port := os.Getenv("APP_PORT")
    envMode := os.Getenv("APP_ENV")
    secretToken := os.Getenv("MERCHANT_SECRET_TOKEN")

    fmt.Println("\n=====")
    fmt.Printf("    MEMULAI SERVER GO-SHOP DI PORT %s (%s)\n", port, envMode)
    fmt.Println("=====")

    // 2. Setup database inventaris dummy
    inventory := map[string]model.Product{
        "SKU-ROG-01": {SKU: "SKU-ROG-01", Name: "Laptop ROG Zephyrus", Price: 25
        "SKU-MOU-02": {SKU: "SKU-MOU-02", Name: "Mouse Wireless Logitech", Price

    }

    // 3. Simulasi Transaksi 1: Autentikasi Gagal
    fmt.Println("\n[TRANSAKSI 1] Kasir mencoba checkout dengan token tidak valid..."
    authenticator := auth.NewAuthenticator(secretToken)

    tokenKasirSalah := "token-palsu-123"
    if !authenticator.ValidateToken(tokenKasirSalah) {
        logger.LogFailure("Akses ditolak: Token merchant kasir tidak cocok!")
    }

    // 4. Simulasi Transaksi 2: Belanja Sukses
    fmt.Println("\n[TRANSAKSI 2] Kasir melakukan checkout belanja dengan token valid"
    tokenKasirBenar := "rahasia-kasir-goshop-2026"

    if authenticator.ValidateToken(tokenKasirBenar) {
        cart := service.NewCartService()

```



```

        // Tambahkan laptop ROG x2 dan mouse x3 ke keranjang
        _ = cart.AddItem(inventory["SKU-ROG-01"], 2)
        _ = cart.AddItem(inventory["SKU-MOU-02"], 3)

        // Lakukan checkout
        orderID, totalPay, errCheckout := cart.Checkout(inventory)
        if errCheckout != nil {
            logger.LogFailure(errCheckout.Error())
        } else {
            // Kurangi stok database inventaris riil (simulasi)
            p1 := inventory["SKU-ROG-01"]
            p1.Stock -= 2
            inventory["SKU-ROG-01"] = p1

            p2 := inventory["SKU-MOU-02"]
            p2.Stock -= 3
            inventory["SKU-MOU-02"] = p2

            // Catat log sukses
            logger.LogSuccess(orderID, totalPay)
        }
    }

    // 5. Simulasi Transaksi 3: Gagal karena stok habis
    fmt.Println("\n[TRANSAKSI 3] Kasir mencoba menjual melebihi sisa stok...")
    if authenticator.ValidateToken(tokenKasirBenar) {
        cart := service.NewCartService()

        // Sisa stok ROG tinggal 3 unit. Mencoba beli 4 unit.
        errStock := cart.AddItem(inventory["SKU-ROG-01"], 4)
        if errStock != nil {
            logger.LogFailure(errStock.Error())
        }
    }

    fmt.Println("\n=====")
}

```

Cara Menjalankan Proyek Praktikum:

Buka terminal Anda di direktori root `/go-shop` dan jalankan perintah:

```
go run main.go
```

Pastikan Anda melihat:

1. Pesan inisialisasi logger `[AUDIT SYSTEM] ...` tercetak di atas sebelum server menyala.
2. Tulisan status transaksi sukses dicetak dengan **warna hijau tebal**.
3. Tulisan status transaksi gagal dicetak dengan **warna merah tebal**.

Sesi 5: Tantangan Modifikasi Praktikum (Tugas Mandiri)

Untuk menguji kelenturan struktur folder kasir e-commerce yang telah Anda buat, selesaikan tugas tantangan berikut:

Tantangan: Paket Ekspedisi Pengiriman (`shipping`)

Tambahkan fitur kalkulasi tarif ongkos kirim (ongkir) e-commerce dalam paket modular baru.

Spesifikasi Tantangan:

1. Buatlah direktori baru `/shipping` langsung di root proyek dan buat berkas `shipping.go` di dalamnya.
2. Di dalam berkas `shipping.go`, buatlah interface `Shipper` yang mendefinisikan metode:
 - `CalculateCost(weightKg float64) float64`
 - `GetCourierName() string`
3. Buat dua concrete struct yang mengimplementasikannya secara implisit:
 - `JNEShipper` (dengan tarif flat Rp 9.000 per kg).
 - `GoSendShipper` (dengan tarif flat Rp 15.000 per kg).
4. Hubungkan paket shipping ini ke dalam method `Checkout` di paket `CartService` agar menerima parameter `Shipper` tambahan dan mengakumulasikan total ongkir ke dalam tagihan belanja.
5. Uji coba modul shipping baru tersebut pada `main.go` menggunakan kurir JNE dan GoSend secara bergantian.